

AGRIAI 2.0 | FORENSIC SOFTWARE EXAMINATION

Sequence and State Diagrams

Technical posters for request order, failure branches, identity and persistence

Volume 06

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Founder, designer and sole creator of AgriAI

Computer Science, University of Ghana

Repository examined: ThaddeusHackz/agriai-v2

Evidence commit: 991457272b7fec548e02c645c5593b930af5ea29

Examination date: 18 August 2026

Classification: Internal technical record

Technical basis

This document is grounded in direct source review, route enumeration, dependency audit, secret pattern scanning, strict TypeScript checking, ESLint, production compilation, PostgreSQL integration tests and local HTTP behavior tests. Live third party AI accounts were not available in the examination environment, so provider success is assessed from code paths and contracts rather than a claim of live production availability.

1. Main chat, Gemini success

Farmer	Chat.tsx	/api/chat	search	Gemini	db
type/send					
----->	POST JSON				
		validate			
		track q/msg			
		search if enabled----->			
		<--sources--			
		build system and history			
		----->			
	<--SSE delta--	<-----chunks-----			
<--render-----					
		persist user and answer----->			
	<--SSE done----				

2. Main chat, complete fallback

```

/api/chat
-> Gemini configured?
  yes -> try preferred model and remaining model names
        any non-empty stream -> provider=gemini
        all fail -> continue
-> Cloudflare configured?
  yes -> POST Llama messages
        non-empty response -> provider=cloudflare
        fail -> continue
-> localAnswer
  -> first knowledge keyword match
  -> first curated regular expression
  -> generic card
  -> provider=local, demo=true, sources=[]
-> persist
-> SSE done

```

3. Search grounding

```

/api/chat -> searchWeb
if TAVILY_API_KEY:
  -> Tavily advanced search
  if HTTP and JSON valid:
    <- answer and up to six sources, demo=false
  else:
    -> keylessSearch
else:
  -> keylessSearch
    -> Wikipedia OpenSearch
    -> DuckDuckGo Instant Answer
    <- combined sources, demo=(no sources)
/api/chat -> contextBlock -> system instruction -> model

```

4. Disease vision

```

Photo -> DiseaseDetector -> data URL -> /api/vision
-> string and size validation
-> detectCropDisease
  if no Gemini client:
    <- FALLBACK_DISEASE, demo=true
  else:
    for preferred vision model plus model waterfall:
      -> prompt + inlineData
      <- model text
      -> extract {...}
      -> JSON.parse
      success -> normalized result, demo=false
      all fail -> failure demo card
<- JSON result -> confidence and treatment UI

```

5. Image Studio

```
Prompt -> /api/generate
-> configured provider exists? no -> 503
-> parse and cap prompt
-> agriculture substring found? no -> 400
-> append photography instruction
-> Cloudflare configured? yes -> Flux
    success -> 200 PNG data URL
    failure -> continue
-> Gemini configured? yes -> image model loop
    success -> 200 image data URL
    failure -> continue
-> 502
```

6. Authentication state machine

```
NO SESSION
POST login
    rate limited -> 429
    malformed -> 400
    invalid -> record failed attempt -> 401
    valid -> create random token -> ACTIVE

ACTIVE
cookie + server Database.sessions row
expired or missing row -> NO SESSION
logout -> remove row and clear cookie -> NO SESSION
remembered browser cookie -> up to seven days
non-remembered browser cookie -> eight hours
server row -> always seven days
```

7. Admin settings mutation

```
Admin or editor -> POST /api/admin/settings
-> getSessionUser
-> allowed top-level key loop
-> shallow merge showSections or chat
-> cap top-level strings
-> settings.updatedAt = now
-> mutate
    -> local atomic write
    -> schedule PostgreSQL save
<- current settings

Next chat request reads new prompt and model immediately.
```

8. Persistence state

```
+-----+
| in-process Database |
+-----+
| mutate callback
v
+-----+
| data/db.json.tmp    |
+-----+
| rename
v
+-----+
| data/db.json        |
+-----+
| 1500ms debounce
v
+-----+
| agriai_state id=1   |
| doc JSONB           |
+-----+
```

Boot hydration runs from PostgreSQL only if data/db.json is absent.

9. Weather state

```
GET /api/weather
-> city map or Accra default
-> OpenWeather current + forecast in parallel
  success -> source=openweather, demo=false
  failure -> Open-Meteo current + daily
    success -> source=open-meteo, demo=true
    failure -> deterministic seedForecast
      -> source=seed, demo=true
-> adviceFor current conditions
```

10. Attack sequence: public history

```
Attacker obtains or guesses sessionId
-> GET /api/history?sessionId=...
<- full matching chat, no ownership check
-> POST /api/history with replacement messages
<- existing chat overwritten
-> DELETE /api/history?sessionId=...
<- chat removed
```

Required fix: server-bound ownership token or route removal.

11. Attack sequence: denial of wallet

```
Bot loop
-> POST /api/generate, /api/vision, /api/transcribe, /api/tts
-> GET /api/health
-> POST /api/chat with webSearch=true
No shared rate limiter
-> concurrent vendor calls
-> quota and compute consumed
-> chat and analytics writes grow full document
```

12. One-page provider ladder

Function	Tier 1	Tier 2	Tier 3
Chat	Gemini model waterfall	Cloudflare Llama	Local knowledge
Search	Tavily	Wikipedia	DuckDuckGo or empty
Vision	Preferred Gemini	Other Gemini names	Demo card
Studio	Cloudflare Flux	Gemini images	HTTP error
Weather	OpenWeather	Open-Meteo	Seed data
Speech input	Browser recognition	Gemini audio	Typed input
Speech output	ElevenLabs	None	Disabled/error
State	Memory	Atomic JSON	PostgreSQL mirror

Authorship and control statement

This volume was prepared, authored and issued solely under the name of Thaddeus Nii Teiko Tagoe. These diagrams can be used as implementation posters, onboarding material and incident tracing aids. The repository records identify Thaddeus Tagoe as the founder, sole designer, developer and maintainer of AgriAI.